

Phase 6 - phase6_report

- [PHASE 6 — FINANCIAL INTELLIGENCE](#)
 - [Real Estate AI Decision Intelligence Platform — Phase Completion Report](#)
 - [1. PHASE STATUS](#)
 - [2. OBJECTIVE](#)
 - [3. INPUTS](#)
 - [4. WHAT WAS BUILT](#)
 - [5. THE MISSING INPUT POLICY IN PRACTICE \(Section 6.7\)](#)
 - [6. A REAL DATA-QUALITY ISSUE THIS PHASE FOUND AND HANDLED](#)
 - [6B. POST-DELIVERY AUDIT FIX — `roi\_total\_holding\_period`](#)
 - [7. DEMO BATCH RESULTS \(n=25, reproducible sample, seed=42\)](#)
 - [8. TESTS EXECUTED / PASSED / FAILED](#)
 - [9. MASTER PROMPT REQUIREMENTS COVERAGE](#)
 - [10. BUSINESS VALUE](#)
 - [11. KNOWN LIMITATIONS \(see also `docs/limitations.md`\)](#)
 - [12. DEPENDENCIES](#)
 - [13. SECURITY CONSIDERATIONS](#)
 - [14. PERFORMANCE CONSIDERATIONS](#)
 - [15. PRODUCTION READINESS](#)
 - [16. NEXT PHASE DEPENDENCIES](#)
 - [17. FINAL STATUS](#)

PHASE 6 — FINANCIAL INTELLIGENCE

Real Estate AI Decision Intelligence Platform — Phase Completion Report

Prepared following the Master A-Z Development, Audit & Integration Prompt, Section 19 (Phase Completion Report), and matching the established code-structure convention from Phases 1/3/4/5 (live PostgreSQL via `db/connection.py`, `src/<topic>/*.py` modules, `generate*_report.py` + `export_results_csv.py` + `generate_charts.py` orchestrators, numbered CSVs, chart PNGs, `test_phase6.py`).

1. PHASE STATUS

PHASE 6 — COMPLETE. A deterministic financial engine (Section 6.3/6.4), scenario analysis (6.5), sensitivity analysis (6.6), and the structured `FinancialResult` contract (6.8) are built, wired into three orchestrators (Markdown report, numbered CSV export, charts), and covered by a 25-test automated suite (23 original + 2 audit-fix regression tests) — **25/25 passing**.

2. OBJECTIVE

Translate property facts and assumptions into deterministic investment economics (Master Prompt §6.1): NOI, yields, cap rate, ROI, IRR, NPV, DSCR, cash-on-cash return, payback period, and terminal value — using plain arithmetic, never LLM reasoning or ML prediction, per §3.4.

3. INPUTS

- **Phase 1** `core.properties` (`asking_price`, `monthly_rent`), `core.expenses` (`property_tax`, `maintenance`, `insurance`, `management_cost`, `renovation_cost`), `core.rentals` (`vacancy_days`, `lease_duration`, `monthly_rent`), `core.transactions` (historical sale prices), `core.market_monthly` (locality-level price history), `core.projects` / `core.localities` (property → locality join path).
- **Phase 4** approved model outputs are *not* consumed yet in this phase's demo batch (the demo runs entirely on OBSERVED data to keep the OBSERVED/ASSUMED boundary clean and auditable); the engine's `FinancialAssumptions.purchase_price` / `monthly_rent` fields are ready to accept a Phase 4 Model A/B prediction as an explicit ASSUMED override once Phase 9 wires it in.

4. WHAT WAS BUILT

Module	Master Prompt Section	Purpose
<code>data_loader.py</code>	6.2	Sources every OBSERVED input (price, rent, expenses, vacancy, locality appreciation) from Phase 1 tables. DB path (production) + CSV path (dev/test), same pattern as Phase 5.
<code>engine.py</code>	6.3, 6.4, 6.7	Deterministic cash-flow schedule + NOI/yield/cap-rate/ROI/IRR/NPV/DSCR/cash-on-cash/payback/CAGR. Implements the Missing Input Policy: nothing required is ever guessed.
<code>scenarios.py</code>	6.5	Base / Upside / Downside, built on documented (not calibrated) stress shocks applied on top of the resolved Base case.
<code>scenarios.py</code> (sensitivity)	6.6	One-driver-at-a-time sensitivity across 6 documented drivers.
<code>batch_runner.py</code>	—	Selects a reproducible demo sample of properties with resolvable OBSERVED data and runs the full pipeline on each.
<code>export_results_csv.py</code>	6.8	6 numbered CSVs (<code>docs/financial_results_csv/</code>).
<code>generate_charts.py</code>	—	4 charts (<code>docs/finance_charts/</code>).
<code>test_phase6.py</code>	6.9	25 automated tests.

5. THE MISSING INPUT POLICY IN PRACTICE (Section 6.7)

Every input the engine needs is resolved in this order, and every resolution is tagged:

1. **Caller-supplied assumption** → tagged `ASSUMED` .
2. **Else, OBSERVED data for this property/locality** → tagged `OBSERVED` .
3. **Else** → the field is `None` and listed in `insufficient_data_fields` . The engine returns `validation_status = "INSUFFICIENT DATA"` for that run rather than defaulting to a plausible-looking number.

Two inputs get slightly different, explicitly-documented treatment because a silent `None` would break every downstream metric even when the missing input is genuinely low-stakes:

- **`vacancy_rate`** : if neither the caller nor the property's own rental history supplies one, it defaults to

0.0 (best case) — labeled `ASSUMED_DEFAULT` and flagged in `insufficient_data_fields`, so the result is clearly an *upper bound*, not a fabricated realistic estimate.

- **appreciation_rate**: if the caller doesn't supply one, the engine looks at the property's locality historical CAGR (`data_loader._locality_appreciation`) — but **only uses it if it passes a documented 20%/yr sanity ceiling** (see Section 6 below). If the only available signal is above that ceiling, the field is `INSUFFICIENT_DATA`, not silently applied.

`FinancingTerms.down_payment_pct` defaults to **1.0 (all-cash)** — leverage is opt-in, never assumed.

6. A REAL DATA-QUALITY ISSUE THIS PHASE FOUND AND HANDLED

Cross-checking every locality's historical price CAGR from `core.market_monthly` (the same data Phase 3's EDA flagged for a locality-price-generator compounding bug) found:

- **Median locality CAGR: 12.1%/yr** — a plausible, strong-but-realistic Indian residential market.
- **13 of 90 localities (14.4%) exceed 30%/yr**, up to **132.75%/yr** (locality_id 54) — not economically plausible for a sustained multi-year CAGR, and consistent with the Phase 3-documented bug rather than a genuine market signal.

`data_loader._locality_appreciation()` now tags any locality above **20%/yr** as `OBSERVED_BUT_UNRELIABLE` with `reliable_for_use_as_default = False`. The financial engine's input resolver (`engine.resolve_inputs`) checks this flag and **refuses to use an unreliable locality CAGR as a default appreciation assumption** — it falls back to `INSUFFICIENT_DATA` for that field unless the caller supplies an explicit `appreciation_rate`. This is covered by two dedicated tests (`test_locality_appreciation_flags_unrealistic_cagr_as_unreliable`, `test_unreliable_appreciation_never_auto_used_as_default`).

This is the Master Prompt §4.6 principle ("do not silently trust a target with runaway compounding") applied one phase later than where it was first documented, to a second consumer of the same underlying data.

6B. POST-DELIVERY AUDIT FIX — `roi_total_holding_period`

An independent review of this phase found that `roi_total_holding_period` was computing a **gross cash multiple** (total cash received ÷ equity invested) instead of a **net return** ($(\text{total cash received} - \text{equity invested}) \div \text{equity invested}$). A hand-checkable case (invest ₹1,000, receive ₹1,100 one year later — a 10% return, matching IRR exactly) was returning **ROI = 110%** instead of the correct **10%**, i.e. overstated by exactly 100 percentage points (one full unit of equity) in every case. This was not caught by the original 23-test suite because no test directly checked `roi_total_holding_period` against a known value.

Fixed in `engine.py::compute_metrics` (one-line change, `roi = (total_return - equity_invested) / equity_invested`), with two new regression tests added (`test_roi_is_net_return_not_gross_cash_multiple`, `test_roi_matches_irr_for_single_period_deal`) — **suite is now 25/25 passing**.

`docs/financial_results_csv/03_financial_metrics.csv` and `04_scenario_comparison.csv` have been regenerated with corrected values (e.g. property 2731: ROI was 223.8%, is now 123.8% over the 5-year hold — consistent with its 17.85%/yr IRR compounded over 5 years). No other metric (yield, cap rate, cash-on-cash, IRR, NPV, DSCR, payback, terminal CAGR) was affected — each was independently hand-verified against known formulas/cases during the same review and found correct.

7. DEMO BATCH RESULTS (n=25, reproducible sample, seed=42)

Sample selection: properties with real `core.expenses` and `core.rentals` history and a locality with a reliable appreciation signal — so this batch demonstrates PASS-status results end-to-end. Portfolio-wide coverage of this bar is reported transparently below; it is *not* 100%, and that gap is a documented limitation, not hidden.

Portfolio coverage stat	Value
Total properties	8,000
With ≥ 1 expense record	4,966 (62.1%)
With ≥ 1 rental record	5,577 (69.7%)
With both (this batch's eligibility bar)	3,564 (44.5%)

Base-scenario results across the n=25 demo sample (all resolved to `PASS` — no assumptions were supplied to the engine; every value came from `OBSERVED` data):

Metric	Min	Median	Mean	Max
Gross yield	2.12%	3.73%	3.53%	4.28%
Net yield / cap rate	1.78%	3.26%	3.04%	3.85%
Cash-on-cash return	1.66%	3.05%	2.84%	3.60%
IRR	-1.87%	12.47%	10.15%	18.54%
NPV (at 10% discount rate)	-₹6.00M	₹1.92M	₹1.45M	₹11.09M
Payback period (years)	4.41	4.50	4.57	4.98
Terminal value CAGR	-2.29%	11.86%	9.56%	18.66%

Reading these honestly: net rental yields in this dataset are low (median 3.26%) relative to what a leveraged deal typically needs to service debt comfortably — this is why `test_dscr_is_numeric_when_leveraged`'s example case (20% down, 9% interest) produces a **DSCR of 0.23**, well under the conventional 1.20+ lending threshold. This is not a bug in the engine; it is what the underlying rent/price ratios in this dataset actually imply for a leveraged acquisition. See `docs/finance_charts/cap_rate_vs_coc.png`.

8. TESTS EXECUTED / PASSED / FAILED

25/25 passed (23 original + 2 added during the post-delivery audit – see Section 6B). Coverage: `OBSERVED`-tagging correctness, the unreliable-appreciation guard (end-to-end, not just at the data-loader level), the Missing Input Policy (`INSUFFICIENT DATA` is actually returned, not a guess), NOI arithmetic identity, DSCR None-when-unlevered / numeric-when-levered, IRR against a hand-computable known case (-1000, +1100 $\rightarrow$ 10%) and the no-sign-change-returns-None case, NPV-equals-zero-at-IRR identity, payback period (both a known-value case and a never-recovers case), scenario monotonicity (Downside $\leq$ Base $\leq$ Upside on both IRR and NPV), scenario/sensitivity correctly refusing to run on an unresolvable Base case, and sensitivity direction (higher purchase price $\rightarrow$ monotonically lower IRR).

9. MASTER PROMPT REQUIREMENTS COVERAGE

Section	Requirement	Status
6.3	Income / expense / cash-flow deterministic engine	✓ <code>engine.py</code>
6.4	Gross/net yield, NOI, cap rate, ROI, IRR, NPV, cash-on-cash, DSCR, payback, terminal value, CAGR	✓ <code>engine.compute_metrics</code>
6.5	Base / Upside / Downside scenarios	✓ <code>scenarios.run_scenarios</code>
6.6	Sensitivity across purchase price, rent, vacancy, opex, appreciation, holding period	✓ <code>scenarios.run_sensitivity</code>
6.7	Missing Input Policy — never guess	✓ <code>engine.resolve_inputs</code> , tested

6.8	Structured FinancialResult (assumptions / cash flows / metrics / scenarios / sensitivity / validation status / limitations)	✓ <code>engine.run_financial_engine</code>
6.9	Testing (cash-flow signs, dates, ROI/IRR/NPV/NOI, leverage, DSCR, scenarios, sensitivity)	✓ <code>test_phase6.py</code> , 25/25
3.2	Never invent financial assumptions	✓ enforced by resolver + unreliable-appreciation guard
3.4	Deterministic (non-LLM) calculation	✓ pure Python arithmetic throughout
13	OBSERVED / ASSUMED / DERIVED labeling	✓ <code>governance</code> block in every result

10. BUSINESS VALUE

- Any property with sufficient Phase 1 data now gets a full, auditable investment-economics workup (yield, cap rate, IRR, NPV, DSCR, payback) computed the same deterministic way every time.
- The Downside/Base/UpSide spread and the per-driver sensitivity table give a caller (eventually Phase 9's Finance Agent) a defensible way to say *which* input a deal is most exposed to, not just a single point estimate.
- The unreliable-appreciation guard prevents a known data-quality issue from silently corrupting exit-value and IRR figures for ~14% of localities.
- The 44.5% OBSERVED-data coverage stat is itself a decision-relevant fact: for the other 55.5% of the portfolio, a caller must either supply explicit assumptions or accept an `INSUFFICIENT DATA` result — Phase 9's Decision Engine gate (\$9.7) can use `financial_completeness` exactly as this phase reports it.

11. KNOWN LIMITATIONS (see also `docs/limitations.md`)

1. **Financing terms are never in Phase 1 data** — every leveraged-deal input (down payment %, interest rate, loan term) must be explicitly supplied by a caller; the engine defaults to all-cash rather than guessing typical loan terms.
2. **`purchase_price` uses `asking_price`, not a confirmed transaction price**, unless the caller overrides it. `core.transactions` history for the *same* property is surfaced in `observed_transaction_history` for a caller to use instead where it exists, but is not auto-substituted (asking price and eventual sale price are legitimately different facts; substituting one for the other silently would be a governance violation, not a convenience).
3. **~14% of localities have an unreliable historical appreciation signal** (inherited from the Phase 3-documented data-generator bug) and are excluded from default-appreciation resolution; a caller must supply `appreciation_rate` explicitly for those localities.
4. **Only 44.5% of the property portfolio has both expense and rental history** needed to fully resolve NOI/yield/cap-rate without caller-supplied assumptions; the remaining 55.5% will return `INSUFFICIENT DATA` (or `PASS WITH LIMITATIONS` with only partial OBSERVED inputs) unless a caller supplies the missing pieces.
5. **Scenario stress magnitudes (\$6.5) are documented assumptions, not statistically calibrated** — no historical volatility model exists in this dataset to calibrate them from. Phase 7's Monte Carlo engine is where a calibrated-vs-uncalibrated distinction belongs (\$7.7); Phase 6's three scenario points are explicitly not probabilities of anything.
6. **DSCR is only computed when a loan is present** (`down_payment_pct < 1.0`); it is `None` , not 0 or infinity, for all-cash deals.

12. DEPENDENCIES

Phase 1 (`core.properties` , `core.expenses` , `core.rentals` , `core.transactions` , `core.market_monthly` , `core.projects` , `core.localities`). No dependency on Phase 2-5 outputs for this phase's own calculations (Phase 5 evidence *can* optionally cite Phase 6 outputs later via Phase 9's Finance Agent, per the architecture diagram, but Phase 6 does not depend on Phase 5).

13. SECURITY CONSIDERATIONS

No hardcoded credentials; `db/connection.py` reused unchanged from Phase 3/5, requiring `DB_USER` / `DB_PASSWORD` as environment variables with no defaults. No external network calls. No user-supplied strings are interpolated into SQL (all queries in `data_loader.py` use parameterized `%s` placeholders).

14. PERFORMANCE CONSIDERATIONS

Pure in-memory arithmetic per property; the demo batch of 25 properties (Base+Upside+Downside+6-driver sensitivity, 5 points each = 30 engine calls per property, 750 total engine calls) runs in well under a second. Not measured at full-portfolio scale (8,000 properties) in this phase; no claim is made about batch runtime beyond the demo sample.

15. PRODUCTION READINESS

Per Section 24: **Production-Oriented Prototype**. The calculation engine, missing-input governance, and scenario/sensitivity logic are complete and tested. Recommended before Phase 9 integration: (a) re-run the CSV/chart/report orchestrators with `--source db` (default) against the user's live PostgreSQL instance — this sandbox has no PostgreSQL available, so this run used `--source csv`, the documented dev/test path reading the identical rows Phase 1 loaded; (b) decide, with the business, the actual policy for the 55.5% of the portfolio lacking full expense/rental history (supply portfolio-level default assumptions explicitly, or accept `INSUFFICIENT DATA` for those properties in Phase 9).

16. NEXT PHASE DEPENDENCIES

Phase 7 (Risk, Scenario & Monte Carlo Intelligence) must reuse this phase's `engine.py` unchanged (§7.2: "do NOT duplicate financial calculations") — Monte Carlo sampling should generate distributions over `FinancialAssumptions` fields and feed each sample through `run_financial_engine`, not reimplement NOI/IRR/NPV math. Phase 9's Finance Agent should call `engine.run_financial_engine` / `scenarios.run_scenarios` / `scenarios.run_sensitivity` directly.

17. FINAL STATUS

PHASE 6: COMPLETE. Deterministic financial engine, scenarios, sensitivity analysis, and the structured `FinancialResult` contract all built and tested against the real Phase 1 dataset — 25/25 automated tests passing (including 2 added during a post-delivery audit that found and fixed an ROI calculation bug – see Section 6B), one real data-quality issue (unreliable locality appreciation, ~14% of localities) found and guarded against rather than silently propagated. Ready for Phase 7 (must reuse this engine) and Phase 8 (parallel), then Phase 9 integration.